Woxsen University

School of Technology



A

PROJECT REPORT

on

QUANTUM ENHANCED AI MOMENTUM TRADING IN FINANCIAL MARKETS AND STRATEGIES

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

B. Rohith (Roll Number)
C. Srinish (Roll Number)
K. Chinmai Datha (Roll Number)
K. Shreenandh (Roll Number)
M. Satvik Kumar (Roll Number)

Under the guidance of:

Dr. Ram Soorat
Designation, School of Technology

Department of Artificial Intelligence and Machine Learning
Woxsen University, Hyderabad · October 2026

CERTIFICATE

This is to certify that the project report entitled “Quantum Enhanced AI Momentum Trading in Financial Markets and Strategies” submitted by B. Rohith (Roll Number), C. Srinish (Roll Number), K. Chinmai Datha (Roll Number), K. Shreenandh (Roll Number) and M. Satvik Kumar (Roll Number) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning from Woxsen University is a bonafide record of work carried out by the students under my supervision and guidance.

The work embodied in this project report has been carried out by the candidates and has not been submitted elsewhere for a degree.

Signature of Mentor

Name: Dr. Ram Soorat

Designation: Designation

Date:

Signature of the Head of the Department

Name: Head of Department Name

Department of Artificial Intelligence and Machine Learning

Date:

DECLARATION OF THE CANDIDATES

We hereby declare that the project work entitled “Quantum Enhanced AI Momentum Trading in Financial Markets and Strategies” submitted to the School of Technology, Woxsen University, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning is our original work and has been carried out under the guidance of Dr. Ram Soorat.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student
Name: B. Rohith
Roll Number: Roll Number
Date:

Signature of Student
Name: C. Srinish
Roll Number: Roll Number
Date:

Signature of Student
Name: K. Chinmai Datha
Roll Number: Roll Number
Date:

Signature of Student
Name: K. Shreenandh
Roll Number: Roll Number
Date:

Signature of Student
Name: M. Satvik Kumar
Roll Number: Roll Number
Date:

ACKNOWLEDGMENT

We thank our project guide, Dr. Ram Soorat, Designation, for guidance through every review of this project. A question raised at those reviews, how we would know whether a backtest result was real rather than how good it looked, shaped the testing protocol that became the core of this work.

We are grateful to Head of Department Name, Head of the Department of Artificial Intelligence and Machine Learning, and to the School of Technology, Woxsen University, for the facilities and support provided during the project.

This project builds on open-source work. We thank the TradingView community authors whose indicator scripts we ported and combined: jdehorty (Machine Learning: Lorentzian Classification), LazyBear (WaveTrend Oscillator and Squeeze Momentum), QuantAlgo (Nadaraya-Watson Trend), and the author of the Weis Wave Volume script. We also thank Binance, Bitstamp and Yahoo Finance for the public market data used in the study, and the maintainers of NumPy, pandas, Numba, SciPy, pytest and backtesting.py.

We used Anthropic's Claude Code to run the tuning agents described in Chapter 3. The protocol, the rules the agents worked under and the checks on their output were designed by the team.

Finally, we thank our families and friends for their patience and support throughout the semester.

B. Rohith, C. Srinish, K. Chinmai Datha, K. Shreenandh, M. Satvik Kumar

ABSTRACT

Most individual traders lose money, and most trading strategies that look profitable on historical data stop working on new data. This project tackles both problems with a rule-based momentum strategy for Bitcoin and a testing protocol designed to prevent overfitting.

The strategy, written in Pine Script v6 for TradingView, assigns five open-source indicators one role each. A Lorentzian-distance k-nearest-neighbour classifier proposes entries, Squeeze Momentum acts as a volatility regime gate, a Nadaraya-Watson kernel regression sets the trend direction, the WaveTrend oscillator blocks late entries and Weis Wave Volume confirms that volume supports the move. A trade is taken only when all five agree on a closed candle, with ATR-based stop-loss and take-profit orders.

To run controlled experiments we rebuilt the strategy and TradingView's order-fill rules in Python, checked by 88 automated tests that include causality proofs and a cross-check against an independent library. Using 41,199 hourly BTC/USDT candles, a team of AI agents tuned the strategy: a tuner that proposed one change per run without access to price data, an executor, an adversarial critic instructed to assume every result was overfitted, and a tracker that kept a hash-verified log. The budget was capped at 25 parameter sets, and the selection gates were fixed before the first run.

Over 19 runs, in-sample performance (January 2023 to June 2025) improved from a Sharpe ratio of −1.71 and profit factor 0.52 to a Sharpe ratio of 0.36 and profit factor 1.10, with maximum drawdown falling from 4.20% to 1.49%. The selected configuration failed one of the four pre-registered gates (profitability under stress costs), and its Deflated Sharpe Ratio was only 0.19. On 14 months of locked out-of-sample data, opened once, the strategy lost 3.12% (Sharpe −1.93, 125 trades) while Bitcoin fell 28.0%. Applied without re-tuning to four daily markets, it was profitable on Reliance Industries (+6.05%, profit factor 1.47) but not on Bitcoin, NIFTY 50 or HDFC Bank.

The in-sample improvement did not carry over to unseen data. The main contribution is the testing framework, which recorded this honestly, and the evidence that the adversarial critic flagged the selected configuration as fragile before the out-of-sample test.

Keywords: Algorithmic Trading, Momentum, k-Nearest Neighbours, Kernel Regression, Backtesting, Overfitting, Deflated Sharpe Ratio, Multi-Agent Systems, Large Language Model Agents, Pine Script

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Dataset Overview 3
1.6 Scope and Limitations 4
1.7 Organization of the Report 5
2. LITERATURE REVIEW / TECHNOLOGY REVIEW 6
2.1 Momentum and Trend Following 6
2.2 Kernel Regression for Trend Estimation 6
2.3 Nearest-Neighbour Classification 7
2.4 Volatility and Volume Indicators 7
2.5 Backtest Overfitting and Multiple Testing 8
2.6 Multi-Agent AI Systems and Adversarial Review 8
2.7 Evaluation Metrics 8
2.8 Research Gap 9
3. METHODOLOGY 10
3.1 System Overview 10
3.2 Data Preparation 10
3.2.1 Collection and Integrity 10
3.2.2 Data Isolation 10
3.2.3 Daily Market Data 11
3.3 Strategy Design 11
3.3.1 Entry: Lorentzian Classification 12

3.3.2 Regime: Squeeze Momentum 12
3.3.3 Trend: Nadaraya-Watson Kernel Regression 12
3.3.4 Timing: WaveTrend 13
3.3.5 Confirmation: Weis Wave Volume 13
3.3.6 Exits, Risk Control and Position Sizing 13
3.4 Python Backtest Engine 13
3.4.1 Signal Engine 14
3.4.2 A Documented Deviation (D1) 14
3.4.3 Broker Emulator 14
3.4.4 Metrics 14
3.5 Verification of the Engine 15
3.6 Multi-Agent Tuning Protocol 16
3.6.1 Code Review Before Tuning 16
3.6.2 Agent Roles in the Tuning Loop 16
3.6.3 Budget and Pre-registered Selection Rule 17
3.6.4 Stopping Rule, Freeze and Out-of-Sample Test 18
3.7 Multi-Market Transfer Test 18
3.8 Implementation Details 18
4. RESULTS AND DISCUSSION 19
4.1 Baseline: The Untuned Strategy 19
4.2 The Tuning Trajectory 19
4.3 The Selected Configuration 21
4.4 The Critic's Objections to Run 17 23
4.5 Out-of-Sample Test 24
4.6 Transfer to Daily Markets 25
4.7 The Strategy on TradingView 27
4.8 Discussion 28

5. CONCLUSION AND FUTURE WORK 29
5.1 Summary of Contributions 29
5.2 Limitations 29
5.3 Future Work 30
5.3.1 Quantum-Inspired Parameter Search 30
5.3.2 Walk-Forward and Regime-Aware Evaluation 30
5.3.3 Cost-Aware Design 30
5.3.4 Aligning TradingView and Python 30
5.3.5 Paper Trading and Portfolio Extension 30
5.4 Societal Relevance 31
5.5 Concluding Remarks 31
REFERENCES 32
APPENDIX I: SCREENSHOTS OF WORK DONE 35
APPENDIX II: FROZEN PARAMETER SET 36
APPENDIX III: EXTRACT FROM THE FINAL CRITIC VERDICT 38

LIST OF TABLES

Table 1.1: Primary Dataset (BTC/USDT, 1 Hour) 3

Table 1.2: Study Periods 4

Table 1.3: Daily Datasets Used for the Transfer Test 4

Table 3.1: Indicator Roles in the CRB Strategy 11

Table 3.2: Trading Properties and Cost Scenarios 13

Table 3.3: Automated Test Suites 15

Table 3.4: Agent Roles in the Tuning Loop 16

Table 3.5: Pre-registered Selection Gates 17

Table 3.6: Implementation Environment 18

Table 4.1: In-sample Results of All 19 Tuning Runs (P = pass, F = fail) 20

Table 4.2: Inputs Changed by Tuning (Configuration Hash b8376d7d48f5) 22

Table 4.3: Baseline versus Selected Configuration (In-sample, Jan 2023 to Jun 2025) 22

Table 4.4: Robustness Checks on the Selected Configuration 23

Table 4.5: In-sample versus Out-of-sample Performance 24

Table 4.6: Frozen Parameters Applied to Four Daily Markets 25

Table 4.7: TradingView Strategy Tester Results, BTCUSD Daily (6 Oct 2026) 27

Table A.1: Frozen Parameter Set (Run 17) 37

LIST OF FIGURES

Figure 1.1: BTC/USDT Hourly Close and the Three Study Periods 4

Figure 3.1: System Architecture and Methodology Flow 10

Figure 3.2: The Five-Role Entry Decision 12

Figure 3.3: The Tuning Loop 17

Figure 4.1: In-sample Sharpe Ratio of Each Tuning Run 20

Figure 4.2: In-sample Account Value, Baseline versus Run 17 23

Figure 4.3: Out-of-sample Value of $10,000, Strategy versus Bitcoin Buy-and-Hold 24

Figure 4.4: Out-of-sample Strategy Equity under Base and Stress Costs 25

Figure 4.5: Net Return by Market, Default versus Tuned Parameters 26

Figure 4.6: Signal Funnel on Reliance Industries (Daily, 2004–2026) 26

Figure 4.7: CRB Strategy on TradingView (BTCUSD Daily) with Key Stats 27

Figure A.1: TradingView Data Window Showing the Five Role Conditions 35

1. INTRODUCTION

This project builds a rule-based trading strategy for Bitcoin that combines five technical indicators, two of which are statistical learning models, and then tests it as strictly as we could. Most of the effort went into the testing: a Python copy of TradingView's backtest engine, a tuning loop run by AI agents that includes an adversarial critic, and a block of recent data that was locked away during tuning and opened exactly once at the end.

A note on the title. "Quantum Enhanced" describes the long-term direction of the project. The phase reported here builds and validates the classical AI layer. No quantum hardware or quantum algorithm was used, and quantum-inspired optimisation of the parameter search is listed as future work in Section 5.3.

1.1 Background

Momentum is one of the most studied effects in finance: assets that have risen recently tend to keep rising for a while, and assets that have fallen tend to keep falling [1]. The effect also appears in an asset's own past returns, across futures, currencies and equity indices, which is the basis of trend-following funds [2]. A momentum strategy has to answer three practical questions: when has a move started, which way is it going, and is it too late to join.

Retail traders usually answer these questions with technical indicators drawn on a chart. TradingView is the most widely used charting platform for this, and its Pine Script language lets users write indicators and strategies and backtest them in the built-in Strategy Tester [3]. Thousands of indicators are shared openly on the platform. Each one is tuned for a particular job and fires many false signals when used alone.

Bitcoin is a convenient market for this kind of study. It trades around the clock, so hourly data is plentiful (more than 41,000 hourly candles between January 2022 and September 2026), and it has had both strong rallies and deep crashes within a few years, which tests whether a strategy only works in one kind of market.

1.2 Motivation

In September 2024 the Securities and Exchange Board of India reported that 93% of more than one crore individual traders in the equity futures and options segment lost money between FY22 and FY24, with aggregate losses above ₹1.8 lakh crore [4]. The same study found that 97% of the trading profits of foreign portfolio investors and 96% of those of proprietary traders

in FY24 came from algorithmic trading. Systematic, rule-based trading is not a guarantee of profit, but it removes two common causes of loss: decisions driven by fear and greed, and rules that change from one trade to the next.

The second motivation is that building a rule-based strategy is the easy part. Testing it honestly is harder. A strategy with many settings can be adjusted until it fits past prices almost perfectly, and that fit usually disappears on new data. Bailey et al. show that, after enough trials, a high backtest Sharpe ratio is expected even when the strategy has no real edge [5]. AI coding agents make this worse, because they can try hundreds of settings in an afternoon. We wanted to find out whether the same agents, given strict rules and an adversarial reviewer, could be used to tune a strategy without fooling ourselves.

1.3 Problem Statement

The problem addressed in this project is to design an AI-assisted momentum strategy for Bitcoin that only trades when several independent indicators agree, to tune it on historical data without overfitting, and to measure honestly how well it performs on data it has never seen. The main difficulties are:

- Hourly price changes are dominated by noise, so any single indicator produces many false signals.
- Market regimes change. Bitcoin rose more than sixfold between January 2023 and June 2025 and then fell 28% over the following 14 months, so a strategy tuned on one period may meet a very different market next.
- Trading costs (commission and slippage) are large relative to the typical gain on a short-horizon trade.
- The strategy has 72 tunable settings. Even with only three candidate values per setting there are about 2.3×10^{34} combinations, far too many to search, and searching a large space makes overfitting more likely.
- TradingView's Strategy Tester cannot run hundreds of logged, controlled experiments, so a separate, faithful testing environment is needed.

1.4 Objectives

The specific objectives of this project are:

1. To build a Pine Script v6 strategy in which each of five open-source indicators has exactly one role (entry, regime, trend, timing, confirmation), entries are taken only when all five agree, and all signals are computed on closed candles.
2. To build a Python backtest engine that reproduces the strategy and TradingView's order-fill rules, and to verify it with automated tests for correctness and for the absence of look-ahead bias.
3. To tune the strategy on January 2023 to June 2025 data with a multi-agent loop (tuner, executor, adversarial critic, tracker) under a hard budget of 25 parameter sets and selection rules fixed in advance.
4. To evaluate the frozen configuration exactly once on 14 months of unseen Bitcoin data, and without re-tuning on four daily markets, and to report the results whether or not they are favourable.

1.5 Dataset Overview

The primary dataset is the hourly BTC/USDT candle series from Binance spot, downloaded through the public klines endpoint [6]. Table 1.1 summarises it. The single missing hour is left as a gap, as TradingView also has no candle for it. The file's SHA-256 hash is stored and checked before every run, so any change to the data would be detected.

Attribute	Value
Source	Binance spot API, GET /api/v3/klines
Symbol and timeframe	BTCUSDT, 1 hour
Fields	time (bar open, UTC), open, high, low, close, volume
Total candles	41,199
First and last candle	2022-01-01 00:00 UTC to 2026-09-13 15:00 UTC
Missing candles	1 (2023-03-24 13:00 UTC), not filled
Integrity hash (SHA-256)	82423f2f…c44a64f

Table 1.1: Primary Dataset (BTC/USDT, 1 Hour)

The series is split into three periods (Table 1.2 and Figure 1.1). The warm-up year lets every indicator settle before any trade is counted. The in-sample period is the only data the tuning loop could use. The out-of-sample period was physically locked until the final configuration had been frozen.

Period	Dates (UTC)	Candles	Use
Warm-up	2022-01-01 to 2022-12-31	8,760	Indicator warm-up only
In-sample (IS)	2023-01-01 to 2025-06-30	21,887	Tuning (19 runs)
Out-of-sample (OOS)	2025-07-01 to 2026-09-13	10,552	Run once with frozen parameters

Table 1.2: Study Periods

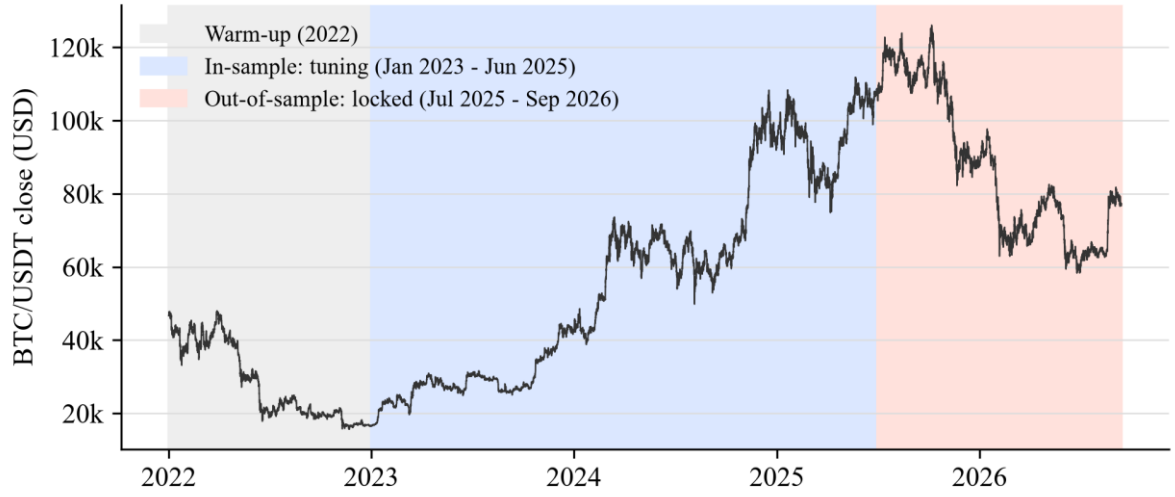


Figure 1.1: BTC/USDT Hourly Close and the Three Study Periods

For the transfer test in Section 4.6 we also used four daily series that the strategy was never tuned on (Table 1.3). In each case the first 2,000 daily candles serve as warm-up, matching the 2,000-bar memory of the nearest-neighbour model.

Market	Source	Daily candles	Evaluation window
Bitcoin (BTCUSD)	Bitstamp API	5,505	2017-02-07 to 2026-09-12
NIFTY 50 index	Yahoo Finance	4,659	2015-11-30 to 2026-09-11
HDFC Bank (NSE)	Yahoo Finance	7,586	2004-01-20 to 2026-09-11
Reliance Industries (NSE)	Yahoo Finance	7,586	2004-01-20 to 2026-09-11

Table 1.3: Daily Datasets Used for the Transfer Test

1.6 Scope and Limitations

The study covers one strategy design, tuned on one market (Bitcoin) at one timeframe (one hour), and evaluated on one locked period plus four daily markets. Its limits are:

- Position size is fixed at 10% of equity per trade, so absolute returns are small by design. Scale-free measures such as the Sharpe ratio and profit factor are the main yardsticks.
- Backtests model commission and slippage but not funding rates, exchange outages or the price impact of large orders.

- With a budget of 25 parameter sets, most of the 72 settings were never changed. The study is a disciplined local search, not a global optimisation.
- The in-app TradingView results depend on how much history TradingView loads, so they cover a shorter window than the Python study (Section 3.4.2).
- No live or paper trading was carried out.

1.7 Organization of the Report

Chapter 2 reviews the literature behind each indicator, research on momentum, the statistics of backtest overfitting and work on multi-agent AI systems. Chapter 3 describes the strategy, the Python engine and its tests, and the agent-based tuning protocol. Chapter 4 presents the tuning results, the out-of-sample test, the multi-market transfer test and the TradingView run. Chapter 5 summarises the contributions and limitations and sets out the next phase of work.

2. LITERATURE REVIEW / TECHNOLOGY REVIEW

This chapter reviews the research each part of the project draws on: momentum and trend following, the two statistical models inside the strategy, the volatility and volume indicators, the statistics of backtest overfitting, and recent work on systems in which AI agents critique one another.

2.1 Momentum and Trend Following

Jegadeesh and Titman found that US stocks that performed best over the previous three to twelve months kept outperforming over the following months [1]. Moskowitz, Ooi and Pedersen later showed a related effect in each asset's own history, which they called time-series momentum: across 58 futures and forward contracts, the sign of the past year's return predicted the next month's return [2]. Hurst, Ooi and Pedersen extended the evidence back more than a century and found trend following profitable in every decade they studied [7].

These studies work at monthly horizons on diversified portfolios. Our strategy trades a single asset on hourly candles, where the effect is weaker and costs matter more. The literature supports the premise that trends persist, but it does not show that a short-horizon version will survive trading costs. That had to be tested.

2.2 Kernel Regression for Trend Estimation

Nadaraya and Watson independently proposed estimating a regression function as a weighted average of nearby observations, with weights given by a kernel function [8], [9]. Applied to a price series, the estimate at time t is

$$NW(t) = \Sigma\, w(i) \cdot price(t - i) \,/\, \Sigma\, w(i), \quad i = 0, 1, \ldots, L$$

where w(i) falls as the lag i grows. With a Gaussian kernel, $w(i) = \exp(-i^2 / 2h^2)$, where h is the bandwidth. A small bandwidth tracks price closely; a large one gives a smoother line that reacts more slowly.

Many Nadaraya-Watson indicators on TradingView use a centred window that includes future bars. These redraw past values as new data arrives ("repainting") and make backtests look far better than live trading would be. The QuantAlgo script we used [10] computes only the causal endpoint estimate shown above, which uses current and past bars and never changes a value once a bar has closed. We use the slope of this line as the trend filter.

2.3 Nearest-Neighbour Classification

Cover and Hart showed that the nearest-neighbour rule, which labels a new point with the label of the closest training example, has an error rate at most twice the Bayes optimal rate as the training set grows [11]. The k-nearest-neighbour (KNN) version takes a vote among the k closest examples. KNN needs no training step and makes few assumptions about the data, which suits financial series whose patterns drift over time.

The Machine Learning: Lorentzian Classification script by jdehorty applies KNN to price data [12]. Each candle is described by up to five normalised features chosen from RSI, WaveTrend, CCI and ADX [13], [14]. Distance between two candles is measured with the Lorentzian metric, the sum over features of $\log(1 + |a_k - b_k|)$. The logarithm reduces the influence of a single extreme feature difference, which matters in markets where large moves distort Euclidean distance. The script searches the last 2,000 candles, takes every fourth one to reduce correlation between neighbours, and keeps the closest k. Their labels are summed into a prediction whose sign gives the direction.

2.4 Volatility and Volume Indicators

John Carter's TTM Squeeze compares Bollinger Bands, which measure volatility through the standard deviation of price [15], with Keltner Channels, which use the average true range [16]. When the Bollinger Bands sit inside the Keltner Channels, volatility is compressed ("squeeze on"). When they move outside, the squeeze has "fired" and a larger move often follows. LazyBear's Squeeze Momentum script implements this idea [17], and we use it only as a regime gate: no new trade is allowed while volatility is compressed.

LazyBear's WaveTrend oscillator smooths the distance between price and its exponential moving average, scaled by average deviation, into two lines, WT1 and WT2 [18]. Values far above zero mean the market is overbought and values far below mean it is oversold. We use WaveTrend for timing: a long entry is refused if WT1 is already above an overbought cap, because the move is likely to be exhausted.

David Weis adapted Richard Wyckoff's method by grouping price into up-waves and down-waves and adding up the volume traded in each wave [19]. A rising wave with growing volume suggests buyers are committed. The Weis Wave Volume indicator gives the direction of the current wave, which we use to confirm that the direction of a proposed trade matches the direction of the current volume wave.

2.5 Backtest Overfitting and Multiple Testing

A backtest is a single draw from a noisy process. If many strategy variants are tried and the best is reported, its measured performance is biased upwards. Bailey, Borwein, López de Prado and Zhu show how quickly this happens: with only a few years of daily data, trying a few dozen configurations is enough to find one with an impressive in-sample Sharpe ratio that has no out-of-sample value [5]. Harvey, Liu and Zhu make the same point about academic factor research and argue that a t-statistic above 3.0, not 2.0, should be required once many tests have been run [20].

Two statistics from this literature are used directly in the project. The Probabilistic Sharpe Ratio (PSR) gives the probability that the true Sharpe ratio is above a benchmark, allowing for track-record length, skewness and fat tails in the returns [21]. The Deflated Sharpe Ratio (DSR) raises that benchmark to the Sharpe ratio expected from the best of N unskilled trials, so that every additional trial makes the test harder to pass [22]. López de Prado also recommends recording every trial, fixing selection rules before testing and keeping a true hold-out set untouched until the end [23]. Our protocol in Section 3.6 follows these recommendations.

2.6 Multi-Agent AI Systems and Adversarial Review

Large language model agents can write code, run experiments and read results, which makes them useful for repetitive research loops. They also tend to accept their own conclusions too readily. Irving, Christiano and Amodei proposed AI safety via debate, in which two agents argue opposite sides so that a judge can spot flaws neither would admit alone [24]. Du et al. found that letting several language model instances critique each other's answers over several rounds improved factual accuracy and reasoning [25], and Madaan et al. showed gains from having a model critique and revise its own output [26].

We applied these ideas to strategy tuning. Instead of one agent that both proposes and judges changes, the tuning loop separates the roles and gives the judging role to an agent instructed to act as a hostile risk committee. The agents were run with Claude Code subagents [27].

2.7 Evaluation Metrics

The Sharpe ratio, the mean excess return divided by the standard deviation of returns, is the standard measure of return per unit of risk [28]. The Sortino ratio penalises only downside deviation [29]. Trading studies also report the profit factor (gross profit divided by gross loss), the win rate, the maximum drawdown (the largest fall from a previous equity peak) and the t-

statistic of mean trade return. A strategy that earns less than buy-and-hold in absolute terms can still be useful if it earns more per unit of risk or carries much less drawdown, so all of these are reported against buy-and-hold on the same window.

2.8 Research Gap

Open-source indicator scripts are usually shared with in-app backtests that do not show how many settings were tried, use repainting calculations or leave out costs. Academic momentum research, on the other hand, rarely deals with the practical details of short-horizon execution. We found little work that combines published community indicators into a role-based strategy and then tests it with multiple-testing corrections, a pre-registered hold-out and an AI tuning loop whose every trial is logged. This project aims to fill that gap at the scale of a student capstone.

3. METHODOLOGY

This chapter describes the data handling, the design of the strategy, the Python engine used to test it, the checks that the engine is correct, and the protocol under which the AI agents tuned it.

3.1 System Overview

The system has two halves (Figure 3.1). The first is the strategy itself, written in Pine Script v6 so that it runs on TradingView. The second is a Python research environment that loads the hourly data, recomputes every indicator exactly as Pine does, simulates TradingView's broker, computes performance metrics and records every experiment in an append-only log. The multi-agent tuning loop works only through this environment. When tuning finished, the chosen parameters were copied back into the Pine script as its default inputs.

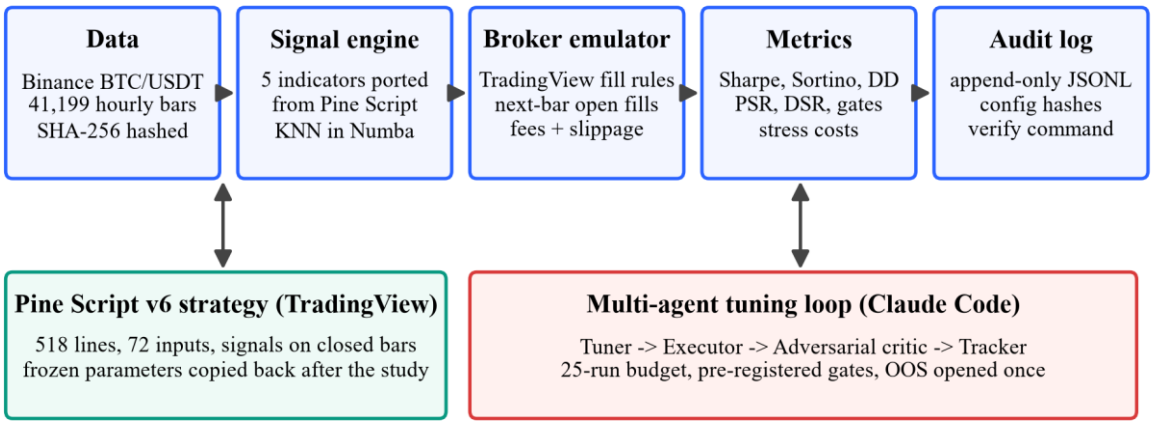


Figure 3.1: System Architecture and Methodology Flow

3.2 Data Preparation

3.2.1 Collection and Integrity

The script `data/fetch_data.py` downloads hourly candles from the Binance klines endpoint in pages, stores the bar open time in UTC and keeps only fully closed candles. It writes a metadata file with the fetch time, row count, first and last candle, the list of gaps and the SHA-256 hash of the CSV file. Gaps are not filled, because filling them would create candles that TradingView does not have. The one missing hour was the only gap found.

3.2.2 Data Isolation

Look-ahead and data leakage were treated as the most serious risks. Three rules were enforced in code, not left to discipline:

1. For any in-sample run, the data frame is cut at 2025-06-30 23:00 UTC before any indicator is computed, so no in-sample calculation can see later prices.
2. Every in-sample run goes through one command, `run_backtest.py is`, which refuses to run if the run number is not the next in sequence, if the 25-run cap is reached, if the same configuration was already tried or if the data hash has changed.
3. The out-of-sample command, `run_backtest.py oos`, writes a LOCK file before it computes anything and refuses to run if that file exists. It can therefore be run only once.

3.2.3 Daily Market Data

For the transfer test, daily candles were downloaded from Bitstamp for Bitcoin (matching TradingView's BITSTAMP:BTCUSD symbol) and from Yahoo Finance for NIFTY 50, HDFC Bank and Reliance Industries. Each series was checked for missing values, impossible OHLC combinations and calendar gaps, and hashed in the same way as the hourly data.

3.3 Strategy Design

The strategy is called the Combined Role-Based (CRB) strategy. Its main design rule is that each indicator does one job only. Table 3.1 lists the roles and Figure 3.2 shows how they combine.

Role	Indicator (author)	Condition for a long entry (short is mirrored)
Entry	Lorentzian Classification (jdehorty)	KNN signal flipped to long within the last 2 bars, is still long, and that flip has not yet produced a trade
Regime gate	Squeeze Momentum (LazyBear)	Squeeze is off: Bollinger Bands are outside the Keltner Channels
Trend	Nadaraya-Watson Trend (QuantAlgo)	NW line higher than 3 bars ago and close above the line
Timing	WaveTrend (LazyBear)	WT1 below the overbought cap (40 after tuning)
Confirmation	Weis Wave Volume	Current Weis wave direction is up

Table 3.1: Indicator Roles in the CRB Strategy



Figure 3.2: The Five-Role Entry Decision

The final entry rule is a logical AND of the five role conditions, evaluated only when the candle has closed. The code below is taken directly from the strategy script.

```
bool barConfirmed = barstate.isconfirmed
bool longCondition  = barConfirmed and allowLongs and regimeGate
     and trendLong and entryLong and timingLong and confirmLong
bool shortCondition = barConfirmed and allowShorts and regimeGate
     and trendShort and entryShort and timingShort and confirmShort
```

3.3.1 Entry: Lorentzian Classification

The KNN model uses five features: RSI(14), WaveTrend(10, 11), CCI(20), ADX(20) and RSI(9), each normalised to the range 0 to 1. The training label for each historical candle compares its close with the close four candles earlier. The model searches the previous 2,000 candles and keeps 12 neighbours after tuning (8 by default). The original script also filters predictions with its own kernel-regression and moving-average filters. We removed those, because trend direction belongs to the Nadaraya-Watson role alone, and kept only the script's volatility and regime filters. To stop one signal from producing several trades, an entry is allowed only within two candles of the flip and only once per flip.

3.3.2 Regime: Squeeze Momentum

Bollinger Bands and Keltner Channels are both computed over 20 candles. The gate opens when the squeeze is off, that is, when volatility has expanded. The script supports two other modes ("not in squeeze" and "released within N bars"). One of these was tested during tuning (run 8) and rejected.

3.3.3 Trend: Nadaraya-Watson Kernel Regression

The trend line uses a Gaussian kernel over the last 50 candles with an effective bandwidth of 5 × 2.0 = 10 after tuning. A long trade needs the line to be higher than it was three candles earlier

and the close to be above it. Requiring price to be on the correct side of the line was one of the earliest useful changes (run 4).

3.3.4 Timing: WaveTrend

By default the timing role required a recent WT1/WT2 crossover and WT1 below a cap of 53. Tuning changed this to the cap alone (run 5) and lowered the cap to 40 (run 17), so the strategy refuses to buy when WT1 is above 40 and refuses to sell short when it is below −40.

3.3.5 Confirmation: Weis Wave Volume

Waves are formed with an ATR(14)-based reversal size, and the current wave direction must match the trade direction. An optional rule requiring the wave's volume to exceed its average was tested in run 9 and rejected.

3.3.6 Exits, Risk Control and Position Sizing

Each entry places a stop-loss two ATR(14) below the entry price and a take-profit three ATR(14) above it for a long trade, and the reverse for a short. Because both levels are set in units of ATR, they widen in volatile markets and tighten in quiet ones. A position is also closed if the KNN signal flips against it or once it has been held for 12 candles. Table 3.2 lists the fixed trading properties, which match the Pine script's strategy() settings.

Property	Base case	Stress case
Initial capital	$10,000	$10,000
Position size	10% of equity per trade	10% of equity per trade
Pyramiding	None (one position at a time)	None
Commission	0.075% per side	0.10% per side
Slippage	2 ticks ($0.02) on market and stop fills	5 basis points on market and stop fills
Stop-loss / take-profit	2 × ATR / 3 × ATR	Same
Maximum holding period	12 candles	Same

Table 3.2: Trading Properties and Cost Scenarios

The stress case is deliberately harsher than Binance's retail fees. A strategy whose profit disappears under it has too little margin over its trading costs.

3.4 Python Backtest Engine

TradingView cannot run large numbers of logged experiments, so we rebuilt the strategy and TradingView's order handling in Python. The engine has four modules.

3.4.1 Signal Engine

`engine/pine_ta.py` re-implements every Pine built-in the strategy uses (sma, ema, rma, rsi, cci, atr, linreg, crossover, barssince and others), matching Pine's handling of missing values and its seeding rules. For example, Pine seeds an EMA with the simple average of its first n values. `engine/mlext.py` ports the feature and filter functions from the MLExtensions library used by the Lorentzian script. The nearest-neighbour search runs once per candle over up to 2,000 previous candles, so it is compiled with Numba [30]. One full in-sample backtest of 21,887 candles takes about 2.4 seconds.

3.4.2 A Documented Deviation (D1)

The original Lorentzian script anchors its search window to the last bar of the chart. On TradingView this means it produces predictions only for the most recent 2,000 candles on the chart, and earlier candles get none. That behaviour cannot be reproduced fairly in a backtest that walks forward through time. The Python engine therefore applies the script's stated intent, a search over the most recent 2,000 candles, at every candle in a strictly causal way. This difference, recorded in the specification as deviation D1, is the main reason why TradingView's own Strategy Tester covers a shorter window than our study (Section 4.7).

3.4.3 Broker Emulator

`engine/broker.py` follows TradingView's strategy semantics candle by candle. Orders created at the close of a candle are filled at the next candle's open, plus adverse slippage. Stop and take-profit levels are checked within each candle along the price path TradingView assumes: if the open is closer to the high, the path is open, high, low, close, and otherwise open, low, high, close. If a candle opens beyond a stop or target level, the order fills at the open. Commission is charged on every fill. An opposite entry closes and reverses the position in one order, as in Pine. Any open position is closed at the end of the test window.

3.4.4 Metrics

`engine/metrics.py` converts the hourly equity curve to daily returns and computes the measures fixed in the specification before tuning began. The main ones are:

- Annualised Sharpe ratio = mean(r) / std(r) × √365, using daily simple returns r with a zero risk-free rate; Sortino ratio uses downside deviation only.

- Maximum drawdown and its duration, Calmar ratio, profit factor, win rate with a 95% confidence interval, average and median trade return, longest losing streak and the t-statistic of trade returns.
- PSR(0), the probability that the true Sharpe ratio is above zero, adjusted for skewness and kurtosis [21].
- DSR across all logged trials, using the variance of Sharpe ratios across trials and the expected maximum of N unskilled Sharpe ratios [22].
- Results by sub-period (2023, 2024 and the first half of 2025), long and short trades separately, the same run under stress costs, and buy-and-hold over the same window.

3.5 Verification of the Engine

A tuning study is only as good as the engine that scores it, so the engine was tested before the first tuning run. The suite has 66 test functions, which expand to 88 test cases through parametrisation. All 88 passed when the study was run and again when this report was prepared. Table 3.3 lists the suites.

Test file	Tests	What it checks
test_ta.py	7	Each Pine built-in against a slow, independent pure-Python loop
test_lc_ann.py	1	Numba nearest-neighbour search against a literal translation of the specification
test_lc_state.py	8	KNN signal state, entry window and one-trade-per-flip rule against a bar-by-bar reference
test_causality.py	2	Signals on a prefix of the data equal signals on the full data; changing a future candle never changes a past signal
test_broker.py	17	Hand-computed fills: next-open entries, intrabar stops, gaps through levels, same-candle stop and target, reversals, window end
test_crosscheck_backtestingpy.py	1	Equity and trades match the independent backtesting.py library on synthetic data
test_metrics.py	11	Sharpe, Sortino, drawdown, PSR, DSR and gates against hand calculations
test_params.py	5	Every Pine input default is present, within bounds and hashed consistently
test_guards.py	14	Run sequencing, 25-run cap, duplicate refusal, in-sample truncation, data-hash check, one-shot OOS lock, tamper detection

Table 3.3: Automated Test Suites

The causality tests matter most. They run the signal engine on the first part of a series and then on the whole series, and check that every signal in the overlap is identical. They also perturb a future candle and confirm that no earlier signal changes. Together these rule out look-ahead bias in the signal engine. The cross-check against backtesting.py [31] confirms that the broker's fills, costs and position sizing agree with an implementation we did not write.

3.6 Multi-Agent Tuning Protocol

3.6.1 Code Review Before Tuning

Two Claude Code agents were defined for the strategy script itself. The builder agent wrote and edited the Pine code under the role rules in Table 3.1 and a fixed list of requirements (Pine v6, commission and slippage set, all thresholds exposed as inputs, signals on confirmed bars only, no repainting estimators). The adversary agent, which could read but not edit files, reviewed each version for repainting, look-ahead bias, unrealistic fills, small samples and signs of overfitting, and reported each issue with its severity and a fix.

3.6.2 Agent Roles in the Tuning Loop

Each tuning iteration passed through four agents in a fixed order (Figure 3.3 and Table 3.4).

Agent	Responsibility	Restrictions
Tuner	Reads the log, leaderboard and all critic verdicts; writes exactly one new parameter set and states its hypothesis and expected effect	May not run backtests or look at price data in any form; must keep values round and change few parameters at a time
Executor	Runs the logged in-sample backtest and reports the metrics	May not edit engine code or configurations
Adversarial critic	Acts as a hostile risk committee: scores risk-adjusted return, drawdown, significance, robustness and costs; lists weaknesses with numbers; demands the next experiment; returns CONTINUE or PLATEAU	Read-only; must assume the result is overfitted; may not praise without numbers
Tracker	Saves the critic's verdict verbatim, re-renders the log and leaderboard, and verifies the log's integrity	Must report a failed integrity check

Table 3.4: Agent Roles in the Tuning Loop

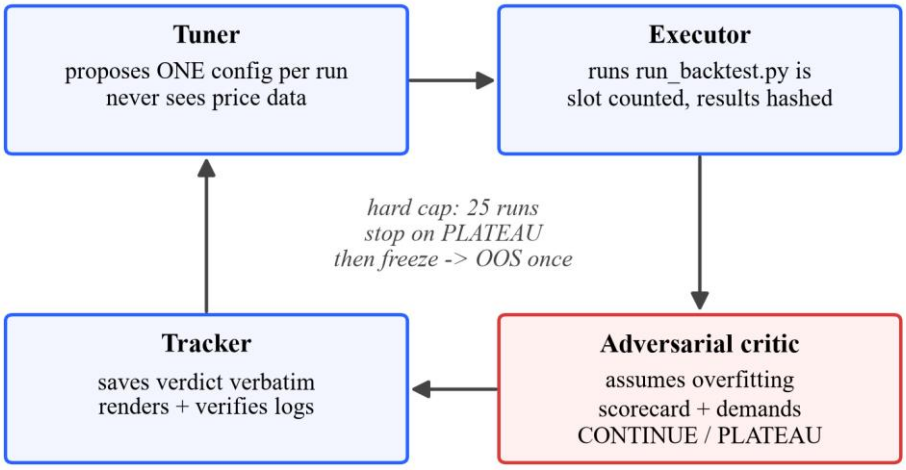


Figure 3.3: The Tuning Loop

The separation is deliberate. The agent that proposes a change never judges it, and the agent that judges it is rewarded for finding faults. Keeping the tuner away from price data stops it from designing a change around patterns it has seen in the chart, so every idea has to be tested through the logged executor and counts against the budget.

3.6.3 Budget and Pre-registered Selection Rule

Before run 1, the following rules were written into the specification and could not be changed afterwards. The budget was 25 in-sample parameter sets, counted by the run command itself. A run was eligible for selection only if it passed all four gates in Table 3.5. The best run was the eligible run with the highest in-sample Sharpe ratio. If no run passed all four gates, the fallback was the highest Sharpe among runs with at least 30 trades, flagged as "FAILED PRE-REGISTERED GATES".

Gate	Rule	Purpose
G1	At least 100 trades	Enough trades for the statistics to mean something
G2	Profit factor above 1.0	Profitable after base costs
G3	Net return above zero under stress costs	Profit margin large enough to survive harsher costs
G4	Net return above zero in at least 2 of the 3 sub-periods (2023, 2024, 2025 H1)	Not dependent on one lucky period

Table 3.5: Pre-registered Selection Gates

3.6.4 Stopping Rule, Freeze and Out-of-Sample Test

The loop stopped when the critic declared a plateau or when run 25 was logged. The critic could declare a plateau only if at least five consecutive runs had failed to raise the best eligible Sharpe by 0.05, every role had been probed, and it judged that further search would be mining noise. After the plateau, a freeze command applied the selection rule and wrote the chosen parameters, with their hash, to `FROZEN_PARAMS.json`. The out-of-sample command was then run once on the full continuous series, with indicators warmed up from 2022 and entries allowed only in the locked window.

3.7 Multi-Market Transfer Test

After the out-of-sample test, the frozen parameters were applied unchanged to the four daily series in Table 1.3 using the same engine. For each market we recorded the signal funnel (how many candidate signals survived each role), base and stress results, long-only results and buy-and-hold. The same markets were also run with the original Pine defaults, so that the effect of tuning on each market could be seen.

3.8 Implementation Details

Component	Tools and versions
Strategy	Pine Script v6 on TradingView; 518 lines, 72 tunable inputs grouped by role
Backtest engine	Python 3.13 with NumPy, pandas, Numba, SciPy and PyArrow; 5,356 lines including tests and the multi-market scripts
Testing	pytest (88 test cases); backtesting.py 0.6.6 for the independent cross-check
AI agents	Claude Code subagents (builder, adversary, tuner, executor, critic, tracker) with written role briefs
Logging	Append-only JSONL log, Markdown log and leaderboard rendered from it, per-run result folders, critic verdict files
Hardware	Windows 11 desktop; about 2.4 seconds per 2.5-year hourly backtest after compilation

Table 3.6: Implementation Environment

4. RESULTS AND DISCUSSION

This chapter reports the tuning results, the selected configuration and the critic's objections to it, the one-time out-of-sample test, the transfer test on four daily markets and the strategy running on TradingView. All figures come from the result files written by the engine. Unless stated otherwise, returns are for a $10,000 account trading 10% of equity per position, after base costs.

4.1 Baseline: The Untuned Strategy

Run 1 used the default inputs of every indicator. Over the in-sample period it lost 3.51% with a Sharpe ratio of −1.71 and a profit factor of 0.52 across 117 trades. Its trade t-statistic was −2.69, so the losses were not bad luck. The short side was the main problem: 66 short trades returned a profit factor of 0.36. Trades were closed after an average of 3.5 candles, mostly by the default 4-candle timer, before any target could be reached, and the take-profit order was switched off. In its first verdict the critic pointed to the short trades fighting a strong bull market, to the 4-candle timer doing almost all of the exit work, and to a payoff ratio below 1 (average loss larger than average win). It demanded probes of the short side, the exits, the stop distance and the trend filter.

4.2 The Tuning Trajectory

Table 4.1 lists all 19 runs and Figure 4.1 shows their in-sample Sharpe ratios. Only five of the 18 changes beat the best result so far (runs 2, 4, 5, 6 and 17). The rest were refuted and recorded, which is what the critic was meant to make happen.

Run	Change	Sharpe	PF	Trades	Net %	Stress %	Gates G1–G4
1	Pine defaults (baseline)	−1.708	0.519	117	−3.51	−5.19	P F F F
2	Hold 12 bars, take-profit on	−0.666	0.787	117	−1.84	−3.40	P F F F
3	Run 2 + stop-loss 3.0 ATR	−0.798	0.747	117	−2.34	−3.89	P F F F
4	Run 2 + NW slope 3, price on trend side	−0.111	0.959	108	−0.30	−1.75	P F F F
5	Run 4 + WT cap only (no cross)	−0.055	0.984	191	−0.19	−2.74	P F F F
6	Run 5 + KNN neighbours 12	0.226	1.060	191	+0.69	−1.87	P P F F
7	Run 6 with neighbours 14	0.182	1.048	193	+0.57	−2.02	P P F F
8	Run 6 + SQZ released-within-window	0.126	1.039	137	+0.33	−1.49	P P F F
9	Run 6 + WWV volume threshold	0.136	1.044	130	+0.33	−1.42	P P F F
10	Run 6 + KNN horizon 8	0.075	1.018	192	+0.22	−2.35	P P F F
11	Run 6 + NW bandwidth 5	0.213	1.053	219	+0.70	−2.25	P P F F
12	Run 11 + 4 KNN features	0.071	1.016	222	+0.23	−2.75	P P F F
13	Run 11 + fixed-hold exit mode	0.116	1.028	219	+0.38	−2.57	P P F F
14	Run 11 + take-profit 2.0 ATR	−0.168	0.960	219	−0.51	−3.30	P F F F
15	Run 11, no take-profit, hold 20	−0.290	0.922	219	−1.23	−4.42	P F F F
16	Run 11 + KNN feature 1 = ADX(50)	0.036	1.007	221	+0.10	−2.86	P P F F
17	**Run 11 + WT caps ±40**	0.355	1.098	188	+1.12	−1.42	P P F P
18	Run 11 + WT caps ±45	0.196	1.051	203	+0.64	−2.10	P P F F
19	Run 17 + WWV part-of-price 400	−0.119	0.964	169	−0.40	−2.66	P F F F

Table 4.1: In-sample Results of All 19 Tuning Runs (P = pass, F = fail)

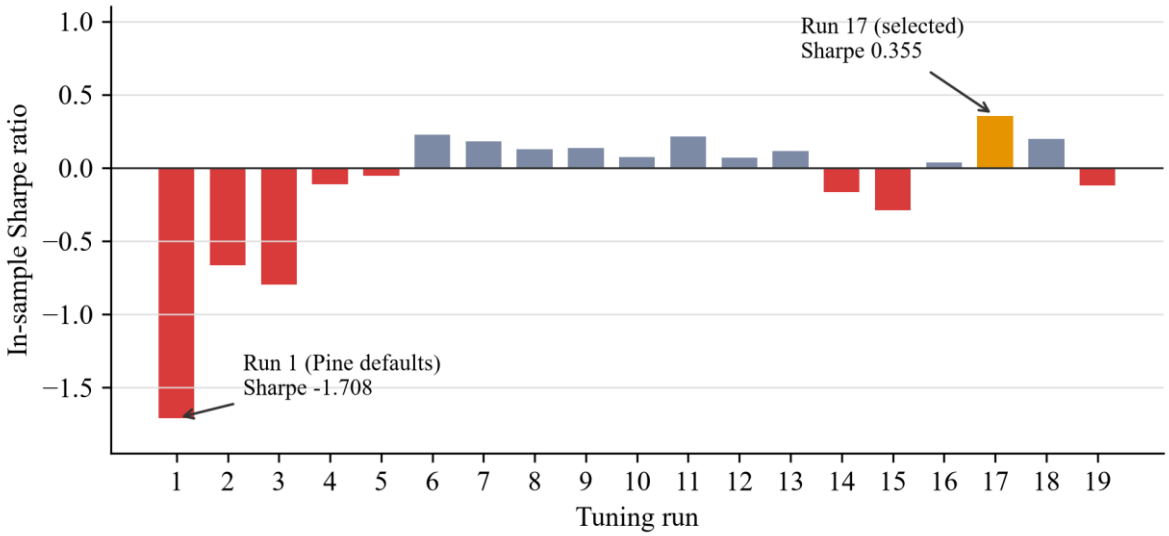


Figure 4.1: In-sample Sharpe Ratio of Each Tuning Run

The runs fall into five phases.

1. **Exits (runs 2–3).** Holding for 12 candles and enabling the 3 ATR take-profit lifted the Sharpe ratio from −1.71 to −0.67. Widening the stop to 3 ATR made it worse.
2. **Trend and timing (runs 4–5).** Requiring a three-candle slope with price on the correct side of the Nadaraya-Watson line, then dropping the WaveTrend crossover requirement, brought the strategy close to break-even.
3. **Entry model (runs 6–7).** Raising the KNN neighbour count from 8 to 12 gave the first positive Sharpe ratio (0.23). A neighbourhood probe at 14 gave 0.18, so the gain was not a single-value spike.
4. **Filters and structure (runs 8–16).** Stricter regime and volume filters, a longer label horizon, fewer features, a different exit mode, tighter and looser targets and a different first feature were all tried. None beat run 6. Several of these runs showed a pattern the critic kept pointing out: filters that only cut the number of trades did not improve the typical trade.
5. **WaveTrend cap (runs 17–19).** Lowering the overextension cap from ±53 to ±40 produced the best Sharpe of the study (0.355). The demanded neighbourhood check at ±45 gave 0.196, lower than both neighbours, and a final volume-wave change gave −0.12. After run 19 the critic declared a plateau, with 6 of the 25 slots unused.

4.3 The Selected Configuration

No run passed all four gates. Every run with a positive Sharpe ratio failed G3, profitability under stress costs, so the pre-registered fallback rule applied. It selected run 17, the highest Sharpe among runs with at least 30 trades, and flagged it “FAILED PRE-REGISTERED GATES”. Run 17 passes G1, G2 and G4 and fails G3 with −1.42% under stress costs. Table 4.2 lists the nine inputs that differ from the indicator defaults. The other 63 inputs were never changed.

Input	Role	Default	Run 17
exitHoldBars	Exit	4	12
riskUseTp	Exit	false	true (3 × ATR)
nwSlopeLen	Trend	1	3
nwRequirePriceSide	Trend	false	true
nwBandwidth	Trend	8	5
lcNeighbors	Entry	8	12
wtTimingMode	Timing	Recent Cross + Not Overextended	Not Overextended Only
wtObLevel2	Timing	53	40
wtOsLevel2	Timing	−53	−40

Table 4.2: Inputs Changed by Tuning (Configuration Hash b8376d7d48f5)

Table 4.3 compares run 17 with the baseline on the in-sample period, and Figure 4.2 shows the two equity curves.

Metric	Run 1 (defaults)	Run 17 (selected)
Net return	−3.51%	+1.12%
Sharpe ratio (annualised)	−1.71	0.36
Sortino ratio	−2.10	0.55
Profit factor	0.52	1.10
Win rate (95% CI)	36.8% (28.6–45.8%)	47.9% (40.8–55.0%)
Average win / average loss	0.90% / −1.00%	1.39% / −1.16%
Maximum drawdown	4.20%	1.49%
Trades (long / short)	117 (51 / 66)	188 (95 / 93)
Profit factor long / short	0.75 / 0.36	1.11 / 1.09
Longest losing streak	9	6
Average holding period	3.5 candles	7.7 candles
Time in market	1.9%	6.6%
Trade t-statistic	−2.69	0.54
PSR (Sharpe > 0)	0.003	0.715
Net return under stress costs	−5.19%	−1.42%
Buy-and-hold BTC, same window	+547.7%, Sharpe 1.79, max drawdown 32.3%	same

Table 4.3: Baseline versus Selected Configuration (In-sample, Jan 2023 to Jun 2025)

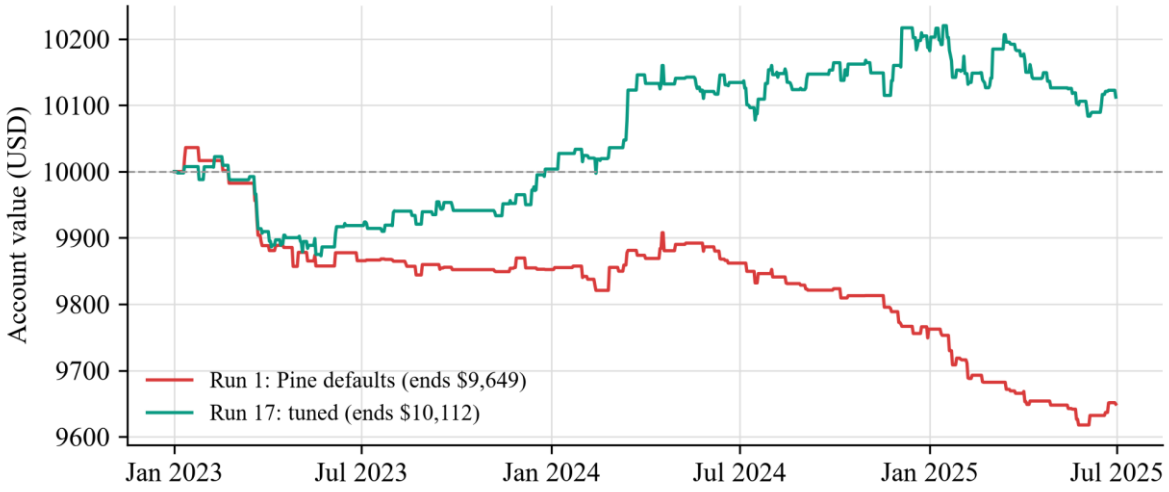


Figure 4.2: In-sample Account Value, Baseline versus Run 17

By every risk measure in the table, tuning turned a strategy that was clearly losing into one that was slightly profitable, with long and short trades both profitable and a lower drawdown. Two numbers in the same table keep this in proportion. A trade t-statistic of 0.54 is far from statistical significance, and buy-and-hold Bitcoin earned a Sharpe ratio of 1.79 over the same window, about five times the strategy's.

4.4 The Critic's Objections to Run 17

The critic did not accept run 17 at face value. Its verdicts on runs 17 and 19 recorded the following problems, each backed by the result files.

Check	Finding
Concentration	The two largest winners ($90.32) made up 80% of the $112.32 net profit. Without the top three trades the net result was −$11.35.
Sub-periods	2023: +0.04% (profit factor 1.01, effectively break-even); 2024: +1.83% (Sharpe 1.27); 2025 H1: −0.73%. The G4 pass rested on a near-zero 2023.
Costs	+1.12% under base costs became −1.42% under stress costs (G3 fail). The average net trade (+0.06%) was smaller than the extra round-trip cost of the stress case (about 0.15 percentage points).
Neighbourhood	The WaveTrend cap went ±53 → ±45 → ±40 with Sharpe 0.21 → 0.20 → 0.36: a dip followed by a spike, the signature of a narrow peak, not a broad effect.
Multiple testing	PSR was 0.715, but the Deflated Sharpe Ratio over 19 trials was only 0.187: about 19% confidence that the Sharpe ratio beats what the best of 19 unskilled trials would be expected to produce.

Table 4.4: Robustness Checks on the Selected Configuration

The critic's final verdict listed these points as caveats that the write-up must carry and said that out-of-sample results should be read "with wide skepticism, not as edge confirmation". That verdict was saved before the out-of-sample data was opened.

4.5 Out-of-Sample Test

The frozen run 17 configuration was run once on the locked period from 1 July 2025 to 13 September 2026. Over this period Bitcoin fell 28.0%, and at its worst point it was 53.7% below its previous peak. Table 4.5 sets the result beside the in-sample figures, and Figures 4.3 and 4.4 show the equity curves.

Metric	In-sample (run 17)	Out-of-sample (run 17, frozen)
Net return	+1.12%	−3.12%
Final account value	$10,112	$9,688
Sharpe ratio	0.36	−1.93
Profit factor	1.10	0.60
Win rate (95% CI)	47.9% (40.8–55.0%)	35.2% (27.4–43.9%)
Trades (long / short)	188 (95 / 93)	125 (64 / 61)
Profit factor long / short	1.11 / 1.09	0.53 / 0.67
Maximum drawdown	1.49%	3.72%
Longest losing streak	6	16
Trade t-statistic	0.54	−2.43
Net return under stress costs	−1.42%	−4.81%
Sub-periods	2023 +0.04%, 2024 +1.83%, 2025 H1 −0.73%	2025 H2 −1.03%, 2026 YTD −2.11%
Buy-and-hold BTC	+547.7% (max drawdown 32.3%)	−28.0% (max drawdown 53.7%)

Table 4.5: In-sample versus Out-of-sample Performance

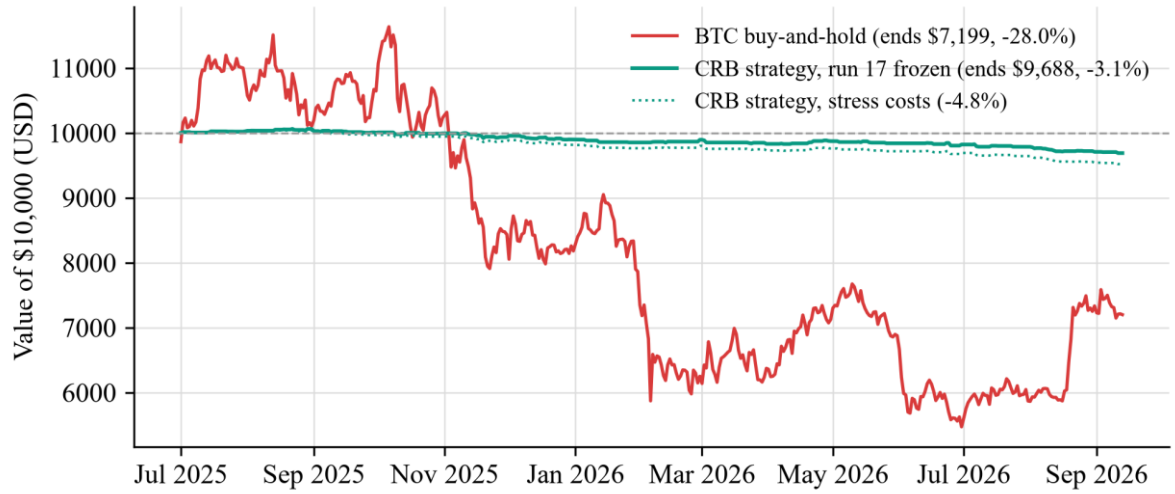


Figure 4.3: Out-of-sample Value of $10,000, Strategy versus Bitcoin Buy-and-Hold

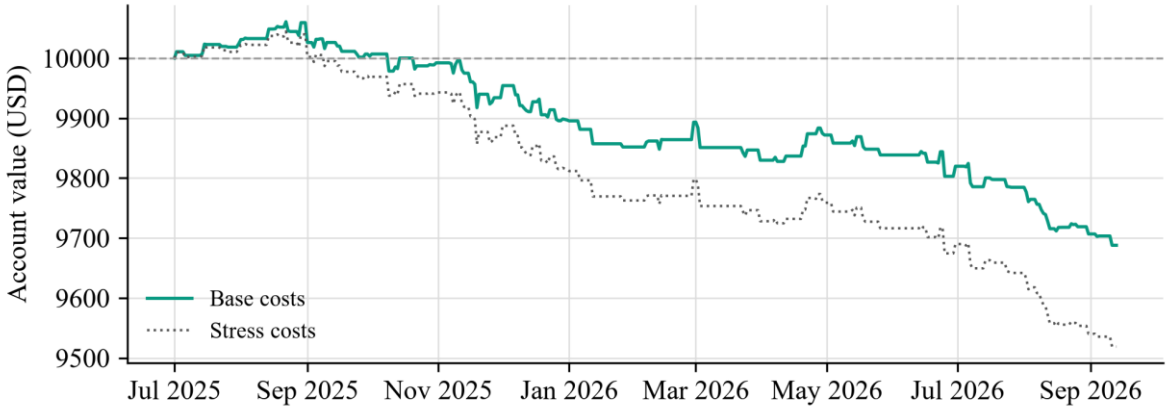


Figure 4.4: Out-of-sample Strategy Equity under Base and Stress Costs

The in-sample edge did not survive. The strategy lost money in both halves of the period, on both long and short trades, and its trade t-statistic of −2.43 means the losses are unlikely to be chance alone. The win rate fell by about 13 points while the ratio of average win to average loss stayed close to its in-sample value (1.10 against 1.20), which suggests that the entries, not the exits, stopped working in the new market.

The comparison with buy-and-hold needs care. The strategy kept 96.9% of its capital while buy-and-hold kept 72.0%, but most of that difference comes from position sizing, not skill. The strategy held a position only 9.7% of the time, each at 10% of equity, so on average less than 1% of the account was exposed to Bitcoin. A passive position of that average size would have lost roughly 0.3% over the period, less than the strategy lost. The risk controls did limit damage (the worst drawdown was 3.7%), but the strategy did not protect capital better than an equally small passive position would have.

4.6 Transfer to Daily Markets

Table 4.6 shows the frozen parameters applied without change to four daily markets, alongside the result with the original defaults. Figure 4.5 shows the same comparison.

Market	Trades	Net % (defaults)	Net % (run 17)	Profit factor	Win rate	Stress net %
Bitcoin 1D (2017–2026)	39	+3.67	−0.43	0.97	51.3%	−0.83
NIFTY 50 1D (2015–2026)	52	−1.32	−1.19	0.81	44.2%	−1.91
HDFC Bank 1D (2004–2026)	58	−0.83	−5.60	0.61	36.2%	−5.83
Reliance 1D (2004–2026)	75	+0.63	+6.05	1.47	58.7%	+5.49

Table 4.6: Frozen Parameters Applied to Four Daily Markets

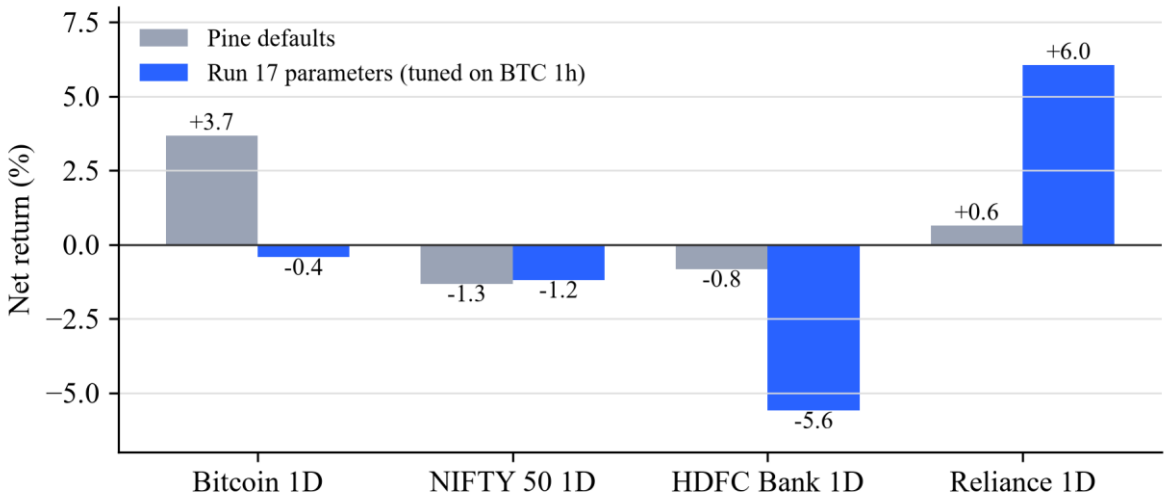


Figure 4.5: Net Return by Market, Default versus Tuned Parameters

Reliance Industries was the one clear success. The tuned settings lifted its 22-year result from +0.63% to +6.05%, with a profit factor of 1.47, a 58.7% win rate and a profit that held under stress costs (+5.49%). On the other three markets the result was flat or negative, and on Bitcoin daily and HDFC Bank the tuned settings did worse than the defaults. One positive market out of four, with a trade t-statistic of 1.28 on that market, is consistent with chance. We report Reliance as an encouraging case, not as evidence that the tuning transfers.

Figure 4.6 shows the signal funnel on Reliance. Of 502 candidate signals from the KNN model, 75 survived all five roles, so 85% were filtered out. The regime gate removed the most (502 to 150), and the trend filter removed most of what remained (150 to 93). The volume confirmation removed only one signal at this stage, which suggests it adds little once the trend filter is in place.

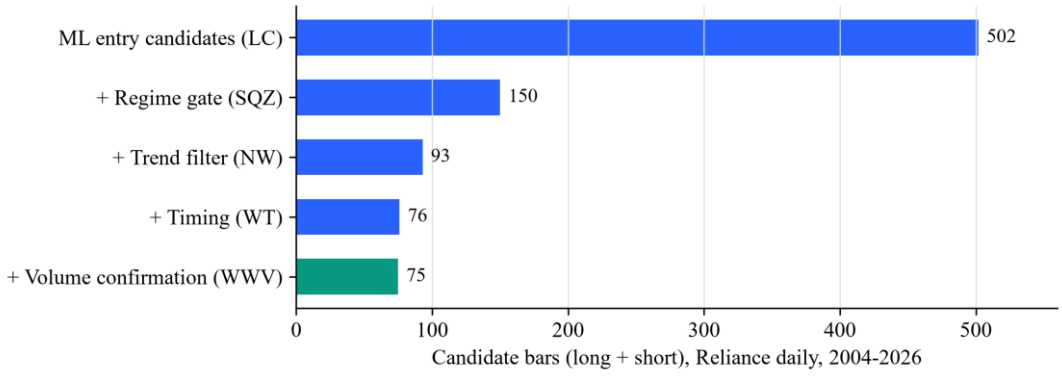


Figure 4.6: Signal Funnel on Reliance Industries (Daily, 2004–2026)

4.7 The Strategy on TradingView

The frozen parameters were written into the Pine script as its default inputs and the strategy was loaded on TradingView. Figure 4.7 shows it on the BTCUSD daily chart with TradingView's own Strategy Tester. The green and red line is the Nadaraya-Watson trend filter, and the line at the top of the chart lists the full parameter set.

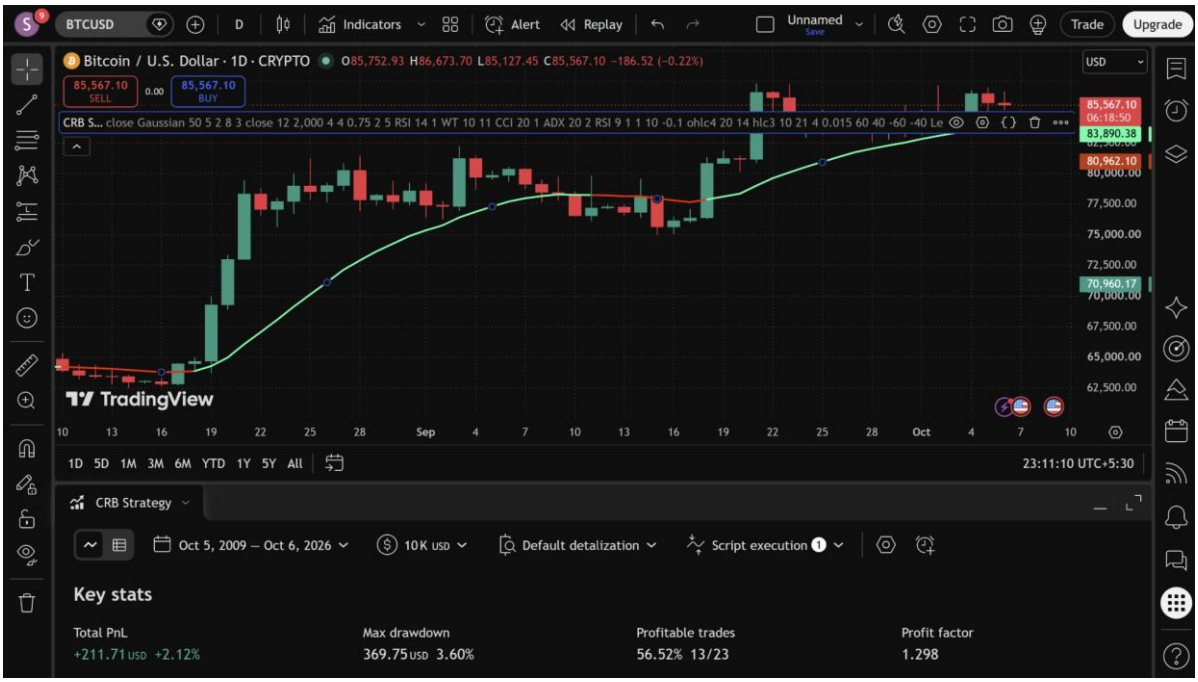


Figure 4.7: CRB Strategy on TradingView (BTCUSD Daily) with Key Stats

Strategy Tester statistic	Value
Total P&L	+$211.71 (+2.12%)
Profitable trades	56.52% (13 of 23)
Profit factor	1.298
Maximum drawdown	$369.75 (3.60%)

Table 4.7: TradingView Strategy Tester Results, BTCUSD Daily (6 Oct 2026)

These numbers differ from the Python daily Bitcoin result in Table 4.6 (39 trades, −0.43%) for the reason given in Section 3.4.2. On TradingView the KNN model produces predictions only over the most recent 2,000 candles of the chart, so the Strategy Tester covers roughly the last five and a half years, not the full history. For the same reason the figures move as new candles arrive. With 23 trades the result is too small a sample to draw conclusions from. What it does show is that the script compiles, runs and trades on TradingView with the tuned settings, and that every entry can be audited: the chart's data window shows each of the five role conditions as 1 or 0 on every candle (Appendix I, Figure A.1).

4.8 Discussion

Three findings stand out.

The process worked even though the strategy did not. Every trial was logged, the selection rule was fixed in advance, and the critic identified the weaknesses of run 17 (concentration in two trades, a near-zero year, a narrow parameter peak, no margin for higher costs and a Deflated Sharpe Ratio of 0.19) before the out-of-sample data was opened. The out-of-sample loss is what those warnings predicted. Without the locked hold-out we would have reported a profitable strategy with a positive Sharpe ratio and a drawdown of 1.5%, and it would have been wrong.

The largest single improvement came from the exits, not from the machine learning model. Letting trades run for 12 candles with a 3 ATR target took the Sharpe ratio from −1.71 to −0.67. Changing the KNN model itself (neighbours, horizon, feature count, feature type) gave smaller gains or none. This matches the momentum literature, which finds that trends are worth riding but not easy to time.

Costs are the binding constraint at this horizon. The selected configuration earned an average of 0.06% per trade after base costs, while a round trip cost about 0.15% in commission alone. Every run with a positive Sharpe ratio failed the stress-cost gate. A short-horizon strategy on Bitcoin needs either a larger edge per trade or fewer, longer trades before it can be trusted.

On the agents themselves: the tuner followed the critic's demands closely, and twice (runs 7 and 18) it spent a slot checking the neighbourhood of its own best result instead of chasing a new high. The critic's stance had to be enforced by its brief. Its default position was that the result was overfitted and that it could not praise without numbers. The team's role was to design these briefs and rules, to check the engine, and to verify the agents' claims against the result files. For run 17, for example, the critic's estimate of the profit concentration was later confirmed by a separate diagnostic script.

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

This project built, tuned and tested an AI-assisted momentum strategy under rules designed to prevent self-deception. Its main contributions are:

1. A role-based Pine Script v6 strategy that combines five open-source indicators, including a Lorentzian KNN classifier and a causal Nadaraya-Watson kernel regression, with every role condition visible on the chart for auditing.
2. A Python backtest engine that reproduces the strategy and TradingView's fill rules, with 88 passing tests including proofs of causality and a cross-check against an independent library, and about 2.4 seconds per 2.5-year hourly backtest.
3. A multi-agent tuning protocol (tuner without data access, executor, adversarial critic, tracker) with a 25-run budget, pre-registered gates, an integrity-checked log and a one-shot out-of-sample lock.
4. An honest record of the outcome: tuning improved the in-sample Sharpe ratio from −1.71 to 0.36, the critic flagged the result as fragile, and the strategy lost 3.12% on unseen data (Sharpe −1.93) while Bitcoin fell 28.0%. Applied to four daily markets without re-tuning, it was profitable only on Reliance Industries (+6.05%).

5.2 Limitations

- The study tuned one market at one timeframe. The out-of-sample period was a single, mostly falling market regime, so it is one test, not a distribution of tests.
- Only 19 parameter sets were tried and 63 of the 72 inputs were never changed. A different search could find a better configuration, but it would also face a higher Deflated Sharpe Ratio hurdle.
- Fixed 10% position sizing keeps absolute returns small and makes drawdown comparisons with buy-and-hold misleading unless exposure is taken into account (Section 4.5).
- Deviation D1 means TradingView's in-app results and the Python study cover different windows, so the two cannot be compared trade for trade.
- The agents' judgments, although checked against the result files, depend on the quality of their briefs. A different critic brief might have stopped the search earlier or later.

5.3 Future Work

5.3.1 Quantum-Inspired Parameter Search

The parameter space has roughly 10^{34} combinations even at three values per input. The next phase of the project is the "quantum enhanced" part of the title: framing the choice of discrete settings as a combinatorial optimisation problem and testing quantum-inspired solvers, such as simulated bifurcation [32] or classical simulations of the Quantum Approximate Optimization Algorithm [33], as search methods. Any such search must keep the same rules as this study, because a more powerful optimiser overfits faster. Every evaluated configuration has to count towards the Deflated Sharpe Ratio, and a fresh, untouched hold-out period is needed, because the current one has already been used.

5.3.2 Walk-Forward and Regime-Aware Evaluation

A single hold-out period gives one data point. Walk-forward testing, which re-tunes on a rolling window and tests on the next block, and combinatorial purged cross-validation [23] would give a distribution of out-of-sample results and show whether the strategy fails only in falling markets.

5.3.3 Cost-Aware Design

Since every positive configuration failed the stress-cost gate, the next designs should target fewer and longer trades, for example on four-hour or daily candles, or use limit orders where fees are lower. Expected cost per trade should become an explicit part of the selection rule.

5.3.4 Aligning TradingView and Python

Rewriting the Lorentzian search in the Pine script so that it is causal at every bar, as in the Python engine, would remove deviation D1 and allow trade-by-trade comparison between TradingView's Strategy Tester and the Python results.

5.3.5 Paper Trading and Portfolio Extension

A forward paper-trading period on live data is the only test that cannot be contaminated by the research process. Running the strategy across a basket of markets with volatility-based position sizing, instead of one market at a fixed 10%, would also show whether the occasional successes, such as Reliance, add up to a useful portfolio.

5.4 Societal Relevance

The project connects to three of the United Nations Sustainable Development Goals. For SDG 4 (Quality Education), the full research record, including specification, code, tests, every tuning run and every critic verdict, can be re-run and used to teach disciplined, evidence-based investing. For SDG 8 (Decent Work and Economic Growth), the strategy design limits each trade to 10% of capital with an automatic stop-loss, and the study shows retail traders why an attractive backtest is not enough evidence to risk money on. For SDG 9 (Industry, Innovation and Infrastructure), it shows a practical way to use AI agents in financial research without letting them overfit.

5.5 Concluding Remarks

We set out to build a strategy that would trade only when five indicators agreed, and to tune it with AI agents without fooling ourselves. The first goal was met. The second was met in the sense that matters: the protocol caught the problem. The tuned strategy looked profitable on the data it was tuned on and lost money on data it had not seen, and the adversarial critic had said in advance that this was likely. A positive in-sample result alone would have been easy to report and difficult to believe. What this project offers instead is a framework that makes it hard to overstate a result. The quantum-inspired search planned for the next phase will have to work under the same rules.

REFERENCES

[1] N. Jegadeesh and S. Titman, “Returns to buying winners and selling losers: Implications for stock market efficiency,” The Journal of Finance, vol. 48, no. 1, pp. 65–91, 1993.

[2] T. J. Moskowitz, Y. H. Ooi, and L. H. Pedersen, “Time series momentum,” Journal of Financial Economics, vol. 104, no. 2, pp. 228–250, 2012.

[3] TradingView, “Pine Script® v6 User Manual,” 2025. [Online]. Available: https://www.tradingview.com/pine-script-docs/

[4] Securities and Exchange Board of India (SEBI), “Updated SEBI study reveals 93% of individual traders incurred losses in equity F&O between FY22 and FY24,” press release and accompanying study, Mumbai, 23 Sept. 2024.

[5] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu, “Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance,” Notices of the American Mathematical Society, vol. 61, no. 5, pp. 458–471, 2014.

[6] Binance, “Spot API documentation: Kline/candlestick data (GET /api/v3/klines),” 2025. [Online]. Available: https://developers.binance.com/docs/binance-spot-api-docs

[7] B. Hurst, Y. H. Ooi, and L. H. Pedersen, “A century of evidence on trend-following investing,” The Journal of Portfolio Management, vol. 44, no. 1, pp. 15–29, 2017.

[8] E. A. Nadaraya, “On estimating regression,” Theory of Probability and Its Applications, vol. 9, no. 1, pp. 141–142, 1964.

[9] G. S. Watson, “Smooth regression analysis,” Sankhyā: The Indian Journal of Statistics, Series A, vol. 26, no. 4, pp. 359–372, 1964.

[10] QuantAlgo, “Nadaraya-Watson Trend,” open-source indicator script, TradingView. [Online]. Available: https://www.tradingview.com/scripts/

[11] T. Cover and P. Hart, “Nearest neighbor pattern classification,” IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[12] jdehorty, “Machine Learning: Lorentzian Classification,” open-source indicator script, TradingView, 2022. [Online]. Available: https://www.tradingview.com/scripts/

[13] J. W. Wilder, New Concepts in Technical Trading Systems. Greensboro, NC, USA: Trend Research, 1978.

[14] D. R. Lambert, “Commodity channel index: Tools for trading cyclic trends,” Commodities, vol. 9, no. 10, pp. 40–41, 1980.

[15] J. Bollinger, Bollinger on Bollinger Bands. New York, NY, USA: McGraw-Hill, 2001.

[16] J. F. Carter, Mastering the Trade: Proven Techniques for Profiting from Intraday and Swing Trading Setups. New York, NY, USA: McGraw-Hill, 2005.

[17] LazyBear, “Squeeze Momentum Indicator [LazyBear],” open-source indicator script, TradingView, 2014. [Online]. Available: https://www.tradingview.com/scripts/

[18] LazyBear, “WaveTrend Oscillator [WT],” open-source indicator script, TradingView, 2014. [Online]. Available: https://www.tradingview.com/scripts/

[19] D. H. Weis, Trades About to Happen: A Modern Adaptation of the Wyckoff Method. Hoboken, NJ, USA: Wiley, 2013.

[20] C. R. Harvey, Y. Liu, and H. Zhu, “…and the cross-section of expected returns,” The Review of Financial Studies, vol. 29, no. 1, pp. 5–68, 2016.

[21] D. H. Bailey and M. López de Prado, “The Sharpe ratio efficient frontier,” Journal of Risk, vol. 15, no. 2, pp. 3–44, 2012.

[22] D. H. Bailey and M. López de Prado, “The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality,” The Journal of Portfolio Management, vol. 40, no. 5, pp. 94–107, 2014.

[23] M. López de Prado, Advances in Financial Machine Learning. Hoboken, NJ, USA: Wiley, 2018.

[24] G. Irving, P. Christiano, and D. Amodei, “AI safety via debate,” arXiv preprint arXiv:1805.00899, 2018.

[25] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” arXiv preprint arXiv:2305.14325, 2023.

[26] A. Madaan et al., “Self-Refine: Iterative refinement with self-feedback,” in Advances in Neural Information Processing Systems, vol. 36, 2023.

[27] Anthropic, “Claude Code documentation: Subagents,” 2025. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code

[28] W. F. Sharpe, “The Sharpe ratio,” The Journal of Portfolio Management, vol. 21, no. 1, pp. 49–58, 1994.

[29] F. A. Sortino and L. N. Price, “Performance measurement in a downside risk framework,” The Journal of Investing, vol. 3, no. 3, pp. 59–64, 1994.

[30] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC, 2015, pp. 1–6.

[31] K. Kernc, “Backtesting.py: Backtest trading strategies in Python,” version 0.6.6, open-source software. [Online]. Available: https://github.com/kernc/backtesting.py

[32] H. Goto, K. Tatsumura, and A. R. Dixon, “Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems,” Science Advances, vol. 5, no. 4, eaav2372, 2019.

[33] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” arXiv preprint arXiv:1411.4028, 2014.

APPENDIX I: SCREENSHOTS OF WORK DONE

Figure A.1 shows the strategy on the BTCUSD one-hour chart with TradingView's data window open. Each role condition is plotted as a separate value (C1 to C5), so any entry can be checked by eye: at the highlighted long entry all five long conditions equal 1.



Figure A.1: TradingView Data Window Showing the Five Role Conditions

The project folder is organised as follows.

```
CIP-Project/
  Combined Role-Based Strategy.pine   Pine v6 strategy, frozen defaults
  5 source indicator .pine files      LC, NW, WT, WWV, SQZ
  .claude/agents/                     builder.md, adversary.md
  backtest/
    SPEC.md, LOOP_PROTOCOL.md         contract, agent role briefs
    data/                             BTCUSDT_1h.csv, meta, fetch
    engine/                           ta, mlext, signals, broker,
                                      metrics, params
    run_backtest.py, log_tools.py     logged CLI, verify, freeze
    tests/                            9 suites, 88 test cases
    configs/run_01..19.json           one parameter set per run
    results/run_01..19/, OOS_FINAL/   metrics, trades, equity
    critic/run_01..19.md              critic verdicts, verbatim
    logs/                             tuning log, leaderboard
    FROZEN_PARAMS.json                selected config and hash
    multimarket/                      daily-market transfer test
```

Insert screenshots of the GitHub repository, and any HackerRank or LeetCode profiles required by the department, here.

APPENDIX II: FROZEN PARAMETER SET

The complete configuration frozen after run 19 (run 17, hash b8376d7d48f5, frozen 2026-09-13 20:02:01 UTC). Inputs changed by tuning are shown in bold. Eight inputs (sources, the CCI constant, the full-history switch and the WWV true-range mode) were fixed by the specification and are not tunable.

Input	Value	Input	Value
allowLongs	true	lcUseAdxFilter	false
allowShorts	true	lcAdxThreshold	20
nwSrc	close	lcAdxLen	14
nwKernelType	Gaussian	wtSrc	hlc3
nwLookback	50	wtN1	10
nwBandwidth	**5**	wtN2	21
nwHMult	2	wtSigLen	4
nwRelWeight	8	wtCciConst	0.015
nwSlopeLen	**3**	wtObLevel1	60
nwRequirePriceSide	**true**	**wtObLevel2**	**40**
lcSource	close	wtOsLevel1	−60
lcNeighbors	**12**	**wtOsLevel2**	**−40**
lcMaxBarsBack	2000	wtExtLevelSel	Level 2
lcIncludeFullHistory	false	**wtTimingMode**	**Not Overextended Only**
lcHorizon	4	wtCrossWindow	5
lcSpacing	4	wwvMethod	ATR
lcDistQuantile	0.75	wwvMethodValue	14
lcEntryWindow	2	wwvPriceSource	Close
lcFeatureCount	5	wwvUseTrueRange	Auto
lcF1String	RSI	wwvNormalize	false
lcF1ParamA	14	wwvUseVolThreshold	false
lcF1ParamB	1	wwvAvgLen	20
lcF2String	WT	wwvAvgMult	1
lcF2ParamA	10	sqzSrc	close
lcF2ParamB	11	sqzBbLen	20
lcF3String	CCI	sqzBbMult	2
lcF3ParamA	20	sqzKcLen	20
lcF3ParamB	1	sqzKcMult	1.5
lcF4String	ADX	sqzUseTrueRange	true
lcF4ParamA	20	sqzReplicateQuirk	true
lcF4ParamB	2	sqzRegimeMode	Squeeze Off
lcF5String	RSI	sqzReleaseWindow	5

Input	Value	Input	Value
lcF5ParamA	9	sqzRequireMomExpansion	false
lcF5ParamB	1	exitMode	Either (LC Default)
lcUseVolFilter	true	**exitHoldBars**	**12**
lcVolMinLen	1	riskUseSl	true
lcVolMaxLen	10	**riskUseTp**	**true**
lcUseRegimeFilter	true	riskAtrLen	14
lcRegimeThreshold	−0.1	riskSlMult	2
lcRegimeSrc	ohlc4	riskTpMult	3

Table A.1: Frozen Parameter Set (Run 17)

APPENDIX III: EXTRACT FROM THE FINAL CRITIC VERDICT

The following caveats are quoted verbatim from the adversarial critic's final verdict (`critic/run_19.md`), written after run 19 and before the out-of-sample test.

1. "Fails G3 — not robust to realistic stress costs (+1.12% base → -1.42% stress)."
2. "Tail-concentrated: PF-ex-top-2=1.019, net-ex-top-2=+$21.99 of $112.32 total (~80% from 2 trades); ex-top-3 net is NEGATIVE (-$11.35)."
3. "The ±40 WT threshold is a non-monotonic, likely fragile local spike (run18 evidence), not a validated broad effect."
4. "G4's pass rests on a near-zero 2023 (+0.037%, essentially coin-flip)."
5. "2025H1 has never been made robustly positive across all 19 runs."
6. "DSR ≈0.18-0.19 at N=19-25 — treat the IS result as statistically unconfirmed after correcting for search. OOS results should be interpreted with wide skepticism, not as edge confirmation."